

# Natív Programozás ZH

SZTE Szoftverfejlesztés Tanszék  
2025. őszi félév

## Technikai ismertető

- A programot C++ nyelven kell megírni.
- A megoldást a *Bíró* fogja kiértékelni.
  - A Feladat beadása felületen a Feltöltés gomb megnyomása után ki kell várni, amíg lefut a kiértékelés. **Kiértékelés közben nem szabad az oldalt frissíteni vagy a Feltöltés gombot újból megnyomni** különben feltöltési lehetőség veszik el!
- Feltöltés után a *Bíró* a programot g++ fordítóval és a  
-std=c++20 -Wall -Werror -static -O2 -DTEST\_BIRO=1  
paraméterezéssel fordítja és különböző tesztesetekre futtatja.
- A program működése akkor helyes, ha a tesztesetek futása nem tart tovább 5 másodpercnél és hiba nélkül (0 hibakóddal) fejeződik be, valamint a program működése a feladatkiírásnak megfelelő.
- A *Bíró* által a `riport.txt`-ben visszaadott lehetséges hibakódok:
  - Futási hiba 6: Memória- vagy időkorlát túllépés.
  - Futási hiba 8: Lebegőpontos hiba, például nullával való osztás.
  - Futási hiba 11: Memória-hozzáférési probléma, pl. tömb-túlinde克斯, null pointer használat.
- A beadások számát és a beadási határidőt a bíróban lehet megtekinteni.
- A programban a `#ifndef TEST_BIRO` és a hozzá tartozó `#endif` közötti rész nem lesz figyelembe véve a kiértékelés során.
- A bíróba egyetlen .cpp forrásfájlt lehet feltölteni (zip fájlt nem fogja kiértékelni).
- Technikai hiba esetén a technikai hiba elhárulása után a hallgató folytathatja a ZH-t, és a ZH határidejét a hibának megfelelően módosítjuk. Ha a hallgató a technikai hiba miatt nem képes folytatni a ZH-t, akkor az adott alkalom igazolt hiányzásnak minősül.
- A ZH megírásához hivatalos fényképes igazolvány szükséges. Ennek hiányában a ZH nem teljesíthető.

## Általános követelmények, tudnivalók

- A programnak követnie kell az objektum-orientált programozás elveit!
- Csak a leírásban szereplő osztályokat, metódusokat és adattagokat kell megvalósítani, egyéb dolgokért nem jár plusz pont.
- Minden metódus, amelyik nem változtatja meg az objektumot, legyen konstans! Ha a paramétert nem változtatja a metódus, akkor a paraméter legyen konstans!
- string összehasonlításoknál az egyezés a pontos egyezést jelenti, azaz ha kis-nagy betűben térnek el, akkor már nem tekinthetők egyenlőnek (pl. a "piros" != "Piros")
- A leírásokban bemutatott példákban a stringek köré rakott idézőjelek nem részei az elvárt kimenetnek, azok csak a string határait jelölik. Például ha az szerepel, hogy a példa bemenetre az elvárt kimenet az, hogy "3 alma", akkor az elvárt kimenet idézőjelek nélkül az 3 alma, de a szóköz szükséges!
  - A tesztesetekben nem lesz ékezetes szöveg kiírása.
- Az elvárt kimeneteknek karakterről karakterre olyan formátumúnak kell lennie, ami a feladatban le van írva (szóközöket és sortöréseket is beleértve).
- Ha az objektum másolása nem triviális (azaz a fordító által generált másolás nem elegendő), akkor a megfelelő másolást is meg kell valósítani.

## Kiindulási projekt, megoldás feltöltése

- **A megoldáshoz az előre kiadott osztályok módosítása szükséges lehet.**
  - Nem minden ZH esetében van kiindulási projekt.
- Feltöltéskor ezeket az osztályokat is fel kell tölteni és a módosításokat is pontozhatja a bíró!
- Egyes tesztesetekben a bíró módosított osztályt is használhat ezen kiinduló osztályok helyett, ezzel tesztelve a valóban helyes működést!

## Zh alatt használható segédanyag

- A ZH során használható segédanyag elérhető bíróban.
  - <https://biro.inf.u-szeged.hu/kozos/native/>

## Dinamikus memória követése

Ebben a feladatban lehetőség van egy speciális konstrukció használatára ami segít az allokált memória követésére. Amennyiben a `new` után egy `TRACK_INFO`-t írunk a `biro` rendszere ki tudja írni, hogy hol került allokálásra az adott memória terület ami nem lett felszabadítva (bizonyos tesztek esetén). Ezt az információt a `teszteset.stdout` kiterjesztésű fájljába írja bele.

Példa használat: `new TRACK_INFO Teglalap(...)`

Példa kimenet az `stdout` fájlban:

```
[ FAIL ] Memory error occured. Here is the map of the allocated memory addresses:
---| FAIL | 0xd7ee40 is not freed. filename: feladat.cpp:123 in function: <function>
---| FAIL | 0xd7eeb0 is not freed. filename: feladat.cpp:123 in function: <function>
---| OK   | 0xd7ef20 is nicely freed.
---| OK   | 0xd7ef50 is nicely freed. filename: feladat.cpp:133 in function: <function>
```

Figyelem! Amennyiben nem `biro`-n kerül fordításra a program, ilyen memória allokáció követés nem történik.

## Kiindulás

**A kiindulási feladatban kapott osztály módosítása vagy bővítése szükséges lehet!**

### Síkidom osztály

- Az `Síkidom` osztály egy síkidomot reprezentál.
- A síkidomnak van egy típusa, amelyet egy `sztring` változóban tárolunk.
- Egy síkidomnak minden oldala biztosan nem negatív egész szám.
- Minden konkrét síkidomnak meghatározható a kerülete, ezt a `kerulet` függvénnyel lehet lekérni. Tovább minden leszármazott osztálynak felül kell definiálnia!
- A `keruletToString` metódus adja vissza a `Síkidom` kerületének értékét az adott formában `"A <típus> kerulete <kerulet> egyseg."`  
Példa: A negyzet kerulete 45 egyseg. **Ez a metódus adott, ezen nem kell változtass!**

## Feladatok

### Teglalap osztály

- A `Teglalap` természetesen egy speciális síkidom, ezért a `Síkidom` osztályból származik.
  - A `Teglalap` az oldalai hosszával legyen adott.
  - Egy téglalapnak két-két szemközti oldala egyenlő hosszúságú.
- Legyen egy két paraméteres konstruktora, ahol megkapja a téglalap két oldalhosszát (jelöljük ezeket `a`-val és `b`-vel).

- Egy téglalap példány típusa pedig `"teglalap"`.
- Egy téglalap területét a négy oldalának az összege adja meg ( $2*a + 2*b$ ).
- Valósítsd meg az osztályt úgy, hogy példányosítani lehessen!

## Negyzet osztály

- A `Negyzet` osztály egy specializált téglalap így származzon a `Teglalap` osztályból!
  - A `Negyzet` egy oldalával legyen adott.
  - Egy négyzetnek mind a négy oldala azonos méretű.
- Legyen egy egy paraméteres konstruktora, ahol megkapja a négyzet oldalhosszát (`a`). A négyzet típusa `"negyzet"`.
- Egy téglalap területét a négy oldalának az összege adja meg ( $4*a$ ).
- Valósítsd meg az osztályt úgy, hogy példányosítani lehessen!

## Haromszog osztály

- A `Haromszog` természetesen ismételtlen speciális síkidom.
  - Egy háromszög a három oldalának hosszával adott.
- Legyen egy három paraméteres konstruktora, ahol megkapja a háromszög három oldalának hosszát (`a`, `b`, `c`). Nem kell vizsgálni, hogy ezen adatokból lehet-e háromszög!
  - A háromszög típusa `"haromszog"`.
- A háromszög területét a három oldalának az összege adja meg ( $a + b + c$ ).
- Valósítsd meg az osztályt úgy, hogy példányosítani lehessen!

## Rajz osztály

- A `Rajz` osztály `Síkidom` típusú objektumokat tárol polimorfikusan `unique_ptr` segítségével, megőrizve az eredeti sorrendjüket.
  - Az osztály lesz a kizárólagos tulajdonosa a `Síkidom`oknak.
- Az osztály rendelkezik default konstruktormal.
- Legyen egy `add` eljárása amivel síkidomokat lehessen az osztályhoz hozzáadni. Az eljárásnak egy paramétere van, ami legyen `Síkidom` és ezt kell eltárolni.
  - Az eljárás legyen láncba fűzhető!
- Az osztálynak legyen egy `dump` eljárása ami egy `std::ostream&`-et kap paraméterül. Erre a kimeneti streamre, soronként kiírja a tárolt síkidomok területét a már meglévő létező `keruletToString` eljárás segítségével. Minden sor végén legyen sortörés!

- Az osztályban tárolt minden síkidom méretét meg lehessen növelni a **increase** eljárással. A méret növelés oly módon legyen megvalósítva, hogy az alakzatok minden oldalának hosszát növeli meg eggyel.
  - A megoldáshoz **for\_each** algoritmust kell használni.
- Legyen az osztálynak másoló (copy) konstruktora!
- Legyen az osztálynak értékadó operátora, de ez legyen különleges olyan értelemben, hogy egy adott rajzból csak a téglalapokat adja értékül az operátor bal oldali kifejezésének! Lehet kell a már előzőleg létrehozott osztályokat is módosítani!